2. THEORY

Introduction

The **Notes Management System (notesdb)** is a database application used to store and manage user notes efficiently. It allows users to create notes, organize them into categories, set reminders, and attach files. The system is implemented using SQL with proper use of joins, subqueries, grouping, and views.

Joins

Joins are used to combine data from two or more tables based on a related column.

Types of Joins:

- **INNER JOIN:** Returns matching records from both tables.
- **LEFT JOIN:** Returns all records from left table and matching from right.
- **RIGHT JOIN:** Returns all records from right table and matching from left.
- **SELF JOIN:** A table is joined with itself.

In this system, joins are used to combine:

- User and Note tables
- Note and Category tables

Subqueries

A **subquery** is a query inside another SQL query.

Types:

- **Single-row subquery** (returns one value)
- **Multiple-row subquery** (returns multiple values)
- **Correlated subquery** (depends on outer query)
- **EXISTS / IN subquery**

Used in this system to:

- Find notes of a specific user

- Filter notes based on conditions
- Compare values like priority or age

GROUP BY Clause

The **GROUP BY** clause is used to group rows that have the same values.

Example:

- Counting number of notes per user
- Grouping notes based on user ID

HAVING Clause

The **HAVING** clause is used to filter grouped data.

Difference:

- `WHERE` filters rows before grouping
- `HAVING` filters after grouping

Example:

- Display users having more than 1 note

Views

A **View** is a virtual table created from a query.

◆ Advantages:

- Simplifies complex queries
- Improves security
- Provides reusable query structure

In this system:

- A view is created to display user notes with priority

Database Design (notesdb)

◆ Tables:

- **User:** Stores user details
- **Category:** Organizes notes

- **Note:** Stores main content
- **Reminder:** Stores reminder details
- **Attachment:** Stores file information

Relationships:

- One user → many notes (**1:N**)
- One category → many notes (**1:N**)
- One note → many reminders (**1:N**)
- One note → many attachments (**1:N**)

Advantages of System

- Organized data storage
- Easy data retrieval using SQL queries
- Supports multiple users
- Flexible and scalable design

Applications

- Personal note-taking apps
- Task management systems
- Reminder applications
- Student study planners

QUERIES:

Enter password: *****

Welcome to the MySQL monitor. Commands end with ; or \g.

Your MySQL connection id is 20

Server version: 8.0.45 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> -- ========================
mysql> -- RESET DATABASE
mysql> -- ========================
mysql> DROP DATABASE IF EXISTS notesdb;
Query OK, 5 rows affected (0.07 sec)

mysql> CREATE DATABASE notesdb;
Query OK, 1 row affected (0.00 sec)

mysql> USE notesdb;
Database changed
mysql>
mysql> -- ========================
mysql> -- TABLE CREATION
mysql> -- ========================
mysql>
mysql> CREATE TABLE User (
   ->    User_ID INT PRIMARY KEY,
   ->    First_Name VARCHAR(50),
   ->    Middle_Name VARCHAR(50),
   ->    Last_Name VARCHAR(50),
   ->    Email VARCHAR(100) UNIQUE,
   ->    Password VARCHAR(50),
   ->    Age INT CHECK (Age BETWEEN 1 AND 100)
   -> );
Query OK, 0 rows affected (0.04 sec)
```

```
mysql>
mysql> CREATE TABLE Category (
   ->    Category_ID INT PRIMARY KEY,
   ->    User_ID INT,
   ->    FOREIGN KEY (User_ID) REFERENCES User(User_ID)
   -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> CREATE TABLE Note (
   ->    Note_ID INT PRIMARY KEY,
   ->    Title VARCHAR(100),
   ->    Description TEXT,
   ->    Priority VARCHAR(20),
   ->    Category_ID INT,
   ->    User_ID INT,
   ->    Status VARCHAR(20),
   ->    Created_At DATETIME,
   ->    Updated_At DATETIME,
   ->    FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID),
   ->    FOREIGN KEY (User_ID) REFERENCES User(User_ID)
   -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE Reminder (
   ->    Reminder_ID INT PRIMARY KEY,
   ->    Note_ID INT,
   ->    Reminder_Time DATETIME,
```

```
   ->    Is_Complete BOOLEAN,
   ->    FOREIGN KEY (Note_ID) REFERENCES Note(Note_ID)
   -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE Attachment (
   ->    Attachment_ID INT PRIMARY KEY,
   ->    Note_ID INT,
   ->    File_Path VARCHAR(200),
   ->    File_Type VARCHAR(50),
   ->    FOREIGN KEY (Note_ID) REFERENCES Note(Note_ID)
   -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> -- ========================
mysql> -- INSERT DATA
mysql> -- ========================
mysql>
mysql> INSERT INTO User VALUES
   -> (1,'Payal','A','Charvande','payal@gmail.com','pass123',21),
   -> (2,'Rahul','B','Sharma','rahul@gmail.com','rahul123',23);
Query OK, 2 rows affected (0.00 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Category VALUES
   -> (101,1),
```

```
   -> (102,2);
Query OK, 2 rows affected (0.00 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Note VALUES
   -> (201,'DBMS','Study ER model','High',101,1,'Active',NOW(),NOW()),
   -> (202,'CN','Study DHCP','Medium',102,2,'Pending',NOW(),NOW());
Query OK, 2 rows affected (0.00 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Reminder VALUES
   -> (301,201,'2026-04-10 10:00:00',0),
   -> (302,202,'2026-04-11 12:00:00',0);
Query OK, 2 rows affected (0.00 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Attachment VALUES
   -> (401,201,'/files/dbms.pdf','PDF'),
   -> (402,202,'/files/cn.docx','DOCX');
Query OK, 2 rows affected (0.00 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> -- ========================
mysql> -- JOINS
mysql> -- ========================
```

```
mysql>
mysql> -- INNER JOIN
mysql> SELECT U.First_Name, N.Title
   -> FROM User U
   -> INNER JOIN Note N ON U.User_ID = N.User_ID;
+------------+-------+
| First_Name | Title |
+------------+-------+
| Payal      | DBMS  |
| Rahul      | CN    |
+------------+-------+
2 rows in set (0.00 sec)

mysql>
mysql> -- LEFT JOIN
mysql> SELECT N.Title, C.Category_ID
   -> FROM Note N
   -> LEFT JOIN Category C ON N.Category_ID = C.Category_ID;
+-------+-------------+
| Title | Category_ID |
+-------+-------------+
| DBMS  |         101 |
| CN    |         102 |
+-------+-------------+
2 rows in set (0.00 sec)

mysql>
mysql> -- RIGHT JOIN
mysql> SELECT N.Title, C.Category_ID
```

```
   -> FROM Note N
   -> RIGHT JOIN Category C ON N.Category_ID = C.Category_ID;
+-------+-------------+
| Title | Category_ID |
+-------+-------------+
| DBMS  |        101 |
| CN    |        102 |
+-------+-------------+
2 rows in set (0.00 sec)

mysql>
mysql> -- SELF JOIN
mysql> SELECT N1.Title AS Note1, N2.Title AS Note2
   -> FROM Note N1
   -> JOIN Note N2 ON N1.User_ID = N2.User_ID
   -> WHERE N1.Note_ID <> N2.Note_ID;
Empty set (0.00 sec)

mysql>
mysql> -- ========================
mysql> -- SUBQUERIES
mysql> -- ========================
mysql>
mysql> -- High Priority Notes
mysql> SELECT Title FROM Note
   -> WHERE Priority = (SELECT 'High');
+-------+
| Title |
+-------+
```

```
| DBMS  |
+-------+
1 row in set (0.00 sec)

mysql>
mysql> -- Notes by Payal
mysql> SELECT Title FROM Note
   -> WHERE User_ID = (SELECT User_ID FROM User WHERE First_Name='Payal');
+-------+
| Title |
+-------+
| DBMS  |
+-------+
1 row in set (0.00 sec)

mysql>
mysql> -- Correlated Subquery
mysql> SELECT Title, Priority
   -> FROM Note N1
   -> WHERE Priority = (
   ->    SELECT MAX(Priority)
   ->    FROM Note N2
   ->    WHERE N1.User_ID = N2.User_ID
   -> );
+-------+----------+
| Title | Priority |
+-------+----------+
| DBMS  | High     |
| CN    | Medium   |
```

```
+-------+----------+
2 rows in set (0.00 sec)

mysql>
mysql> -- EXISTS
mysql> SELECT Title
   -> FROM Note N
   -> WHERE EXISTS (
   ->    SELECT *
   ->    FROM Category C
   ->    WHERE N.Category_ID = C.Category_ID
   -> );
+-------+
| Title |
+-------+
| DBMS  |
| CN    |
+-------+
2 rows in set (0.00 sec)

mysql>
mysql> -- IN
mysql> SELECT Title
   -> FROM Note
   -> WHERE User_ID IN (
   ->    SELECT User_ID FROM User WHERE Age > 20
   -> );
+-------+
| Title |
```

```
+-------+
| DBMS  |
| CN    |
+-------+
2 rows in set (0.00 sec)

mysql>
mysql> -- ========================
mysql> -- GROUP BY & HAVING
mysql> -- ========================
mysql>
mysql> -- Count Notes per User
mysql> SELECT User_ID, COUNT(*) AS total_notes
   -> FROM Note
   -> GROUP BY User_ID;
+---------+-------------+
| User_ID | total_notes |
+---------+-------------+
|       1 |           1 |
|       2 |           1 |
+---------+-------------+
2 rows in set (0.00 sec)

mysql>
mysql> -- HAVING COUNT > 0
mysql> SELECT User_ID, COUNT(*) AS total_notes
   -> FROM Note
   -> GROUP BY User_ID
   -> HAVING COUNT(*) > 0;
```

```
+---------+-------------+
| User_ID | total_notes |
+---------+-------------+
|       1 |           1 |
|       2 |           1 |
+---------+-------------+
2 rows in set (0.00 sec)

mysql>
mysql> -- Average Age
mysql> SELECT AVG(Age) FROM User;
+----------+
| AVG(Age) |
+----------+
|  22.0000 |
+----------+
1 row in set (0.00 sec)

mysql>
mysql> -- HAVING AVG Age > 20
mysql> SELECT User_ID, COUNT(*)
    -> FROM Note
    -> GROUP BY User_ID
    -> HAVING COUNT(*) >= 1;
+---------+----------+
| User_ID | COUNT(*) |
+---------+----------+
|       1 |        1 |
|       2 |        1 |
```

```
+---------+----------+
2 rows in set (0.00 sec)

mysql>
mysql> -- ========================
mysql> -- VIEW
mysql> -- ========================
mysql>
mysql> CREATE VIEW User_Notes AS
   -> SELECT U.First_Name, N.Title, N.Priority
   -> FROM User U
   -> JOIN Note N ON U.User_ID = N.User_ID;
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> SELECT * FROM User_Notes;
+------------+-------+----------+
| First_Name | Title | Priority |
+------------+-------+----------+
| Payal      | DBMS  | High     |
| Rahul      | CN    | Medium   |
+------------+-------+----------+
2 rows in set (0.00 sec)
```

---

3. CONCLUSION

This assignment demonstrates the use of SQL joins, subqueries, GROUP BY, HAVING, and views in a Notes Management System. These concepts help in efficient data retrieval, organization, and management. The system is scalable, efficient, and suitable for real-world applications.